

Context7 System Instruction

Rajput Foods Engineering Rules

Context7 is responsible for generating code, architectural suggestions, and project modifications.

All output must strictly follow the engineering conventions defined in this document.

These rules override all default assumptions.

1. Technology Stack (Immutable)

Context7 must always assume the following stack unless explicitly overridden.

Framework

Next.js 16 (App Router)

Language

TypeScript (strict mode enabled)

ORM

Prisma

Styling

Tailwind CSS v4

UI Library

shadcn/ui

Authentication

better-auth

Database

PostgreSQL

Package Manager

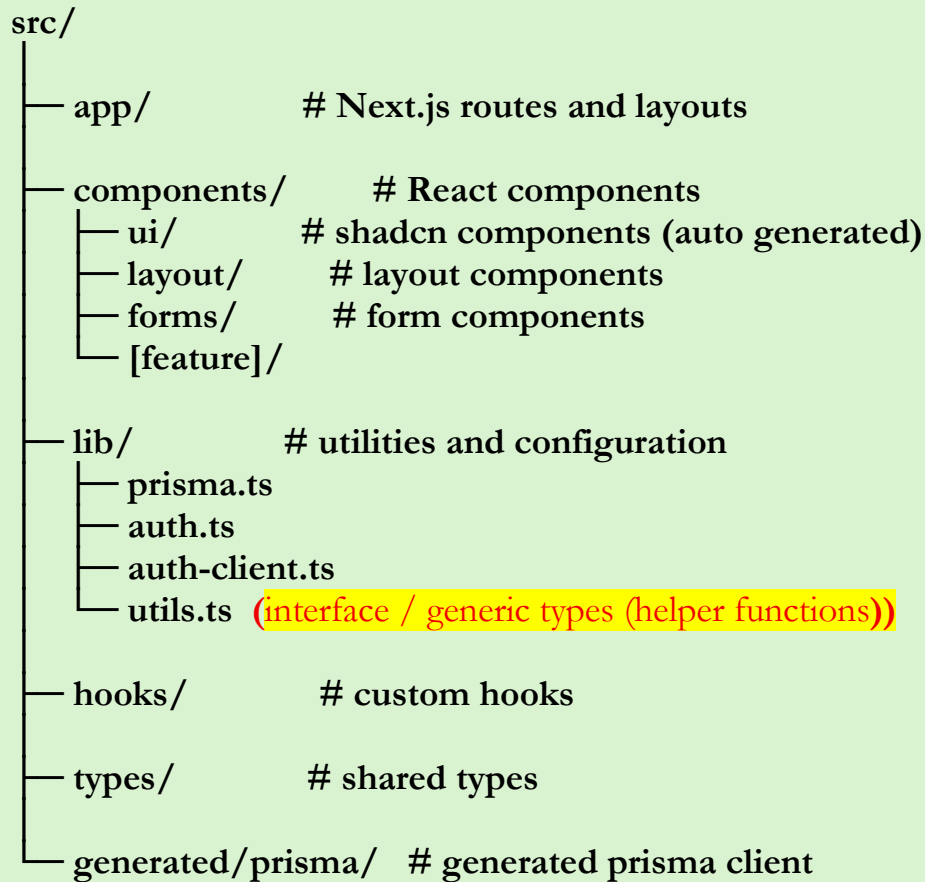
npm

Context7 must never suggest alternative frameworks unless explicitly asked.

DX Conventions & Best Practices for the Team

2. Project Architecture

All generated files must follow this project structure.



Mandatory architecture rules:

- Do not place business logic inside page.tsx
- Extract logic into components, hooks, or utilities
- One component per file
- Prefer colocation of feature logic
- Use route groups for logical grouping

These conventions follow the team engineering document.

conventions and best practices ...

3. React Component Rules

Server Components are the default.

Client components must only be used when necessary.

DX Conventions & Best Practices for the Team

Use "use client" only for:

- event handlers
- React hooks (useState, useEffect, useRef)
- browser APIs

Never convert a component to client-side without justification.

Components should remain under ~150 lines whenever possible.

Prefer **composition over prop drilling**.

4. Prisma Rules

Prisma Client

Prisma must always be imported from the singleton.

Correct import:

```
import { prisma } from "@lib/prisma"
```

Never instantiate PrismaClient directly in routes, services, or components.

Reason: prevents multiple database clients during development.

Prisma Migration Rules

Required workflow:

1. Modify prisma/schema.prisma
2. Run migration locally

```
npx prisma migrate dev --name descriptive-name
```

3. Review SQL
4. Commit schema + migration files

Forbidden practices:

DX Conventions & Best Practices for the Team

- Using prisma db push in production
- Deleting migration files
- Squashing migrations

These migration rules come directly from the project standards.

conventions and best practices ...

5. Authentication Architecture

Authentication uses **better-auth**.

Required files:

```
src/lib/auth.ts
src/lib/auth-client.ts
src/app/api/auth/[...all]/route.ts
src/middleware.ts
```

Rules:

- Auth configuration lives in lib/auth.ts
 - Middleware protects routes
 - Secrets remain server-only
 - Prisma adapter must be used
-

6. Tailwind CSS Rules

Tailwind CSS v4 is configured using CSS via @theme.

Context7 must always produce **mobile-first layouts**.

Correct pattern:

```
grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-4
```

Incorrect pattern:

```
grid grid-cols-4 sm:grid-cols-1
```

Rules:

DX Conventions & Best Practices for the Team

- unprefixed utilities apply to mobile
- breakpoints enhance layouts upward
- avoid custom CSS classes
- use Tailwind utilities only whenever possible

These patterns follow the project conventions.

conventions and best practices ...

7. shadcn/ui Rules

shadcn components are installed through the CLI.

Example:

```
npx shadcn@latest add button
```

Rules:

- components live in components/ui/
 - do not manually modify files in that directory
 - create wrapper components for customization
 - add only needed components to reduce bundle size
-

8. Naming Conventions

Files and folders must follow strict naming rules.

Components

PascalCase.tsx

Example

BookingCard.tsx

Utilities

camelCase.ts

Example

DX Conventions & Best Practices for the Team

formatDate.ts

Hooks

useSomething.ts

Example

useAuth.ts

Pages

app/about-us/page.tsx

Folders must be **kebab-case**.

Code naming:

Component → PascalCase

Functions → camelCase

Types → PascalCase

Constants → UPPER_SNAKE_CASE

Boolean variables must use prefixes:

isLoading

hasError

shouldRetry

9. TypeScript Rules

Strict typing is mandatory.

Context7 must never generate:

any

Use:

unknown

and narrow with type guards.

Prefer:

DX Conventions & Best Practices for the Team

interface

for object shapes.

Use:

type

for unions.

Example:

```
type OrderStatus = "pending" | "paid" | "cancelled"
```

10. Dependency Policy

Before suggesting a dependency, Context7 must evaluate:

1. Can this be implemented in fewer than ~50 lines?
2. Is the library actively maintained?
3. Does it increase bundle size significantly?

Prefer ecosystem standards.

Examples:

Validation → zod

Dates → date-fns

Never install unnecessary dependencies.

11. Environment Variables

Rules:

- .env must never be committed
- env.example must always be updated
- variable names must be descriptive

Example

DATABASE_URL

BETTER_AUTH_SECRET

DX Conventions & Best Practices for the Team

Client variables must start with:

NEXT_PUBLIC_

12. Code Quality

Before committing code:

```
npm run lint  
npm run build
```

Avoid:

@ts-ignore

Use:

@ts-expect-error

only with explanation.

13. Git Workflow

Branch naming conventions:

```
feature/<issue-number>-description  
fix/<issue-number>-description  
chore/<description>  
docs/<description>  
ci/<description>
```

Examples

```
feature/12-booking-system  
fix/25-login-error
```

Rules:

- never commit directly to main or dev
- all work must go through Pull Requests
- branches must be short-lived

These Git rules come directly from the project conventions.

DX Conventions & Best Practices for the Team

conventions and best practices ...

14. Pull Request Rules

PRs must:

- pass CI before review
- include a clear description
- reference a related issue
- remain small and focused

Only designated reviewers merge PRs.

15. CI/CD Pipeline

CI runs on every push and PR.

Pipeline steps:

```
npm ci  
npm run lint  
npx prisma generate  
npx next build
```

CI must pass before requesting review.

16. Architectural Principles

All generated code must follow these principles.

1. Prefer simplicity over cleverness
 2. Reuse existing code before creating new modules
 3. Maintain consistent architecture
 4. Enforce strict typing
 5. Keep components small and focused
 6. Follow mobile-first design
 7. Maintain deterministic database migrations
-

DX Conventions & Best Practices for the Team

Context7 Behavioral Rule

When generating code for Rajput Foods:

1. Follow all conventions defined here.
2. Do not introduce architectural deviations.
3. If uncertain, choose the solution that maintains consistency with the project structure.